Deploying Web Applications on Containers on AWS

Advanced Concepts of Cloud Computing — LOG8415E

Lab Assignment 2

Fall 2026

Abstract

In this lab assignment, you will deploy containerized web applications on AWS using Docker. You will build Flask-based worker containers that process requests, implement a simple orchestrator that distributes incoming requests to available workers, and automate the deployment using scripts and AWS tools. You will document your implementation and results in a LaTeX report.

This version has been adapted for Fall 2026 to reduce infrastructure complexity while preserving the main learning objectives.

1 Introduction

Containerization is a lightweight virtualization technology that packages an application and its dependencies into an isolated unit called a container. Containers help ensure that software behaves consistently across development, testing, and production environments.

In this assignment, you will use Docker to deploy a Flask web application on AWS EC2 instances. Your system will include one orchestrator and two workers. The orchestrator will receive user requests, determine which worker is available, forward the request to that worker, and return the result to the user.

- **Weight:** 100 points.
- **Submission:** One `.zip` file per group on Moodle.
- **Due date:** To be announced on Moodle.

Important billing notice: Stop your EC2 instances when you are not actively working on them. You have limited AWS credits for the course assignments. Manage your resources carefully to avoid running out of credits.

2 Learning Objectives

By completing this assignment, you will be able to:

- Install and use Docker and Docker Compose on Ubuntu EC2 instances.
- Build and run containerized Flask web applications.
- Implement a simple request orchestrator.
- Manage worker availability using a shared JSON status file.
- Queue requests when all workers are busy.
- Automate cloud deployment using Bash, Python, the AWS CLI, or AWS SDKs.
- Document a cloud application architecture and deployment workflow.

3 AWS Lab Requirements

You must complete and test your solution in the AWS Lab environment provided by the course.

- Use the EC2 instance type specified on Moodle. If no type is specified, use `t2.large`.
- Use Ubuntu EC2 instances.
- Deploy exactly **three instances**: one orchestrator and two workers.
- The orchestrator must accept HTTP requests on port `80`.
- Each worker container must listen on port `5000`.
- Configure security groups so that only the required ports are open.
- Do not leave instances running when they are not needed.
- Your complete solution must run in the AWS Lab environment.

4 System Architecture

Your architecture must contain:

- **One orchestrator instance:** Runs a Flask application on port `80`.
- **Two worker instances:** Each runs exactly one Docker container with a Flask application on port `5000`.

You do **not** need to configure multiple containers per worker, an AWS Load Balancer, or target groups.

The workflow is:

1. A user sends a POST request to the orchestrator on port `80`.
2. The orchestrator checks its internal JSON status file.
3. If a worker is free, the orchestrator forwards the request to that worker.
4. The worker processes the request and returns a JSON response.
5. The orchestrator marks the worker as free again.
6. If both workers are busy, the orchestrator places the request in a queue and processes it when a worker becomes available.

5 Setting Up Docker

Install Docker Engine on your Ubuntu instances using the official Docker documentation. Use the **Install using the Apt repository** method.

Use Docker Compose to build and run your Flask application inside a container. Your worker containers must expose port `5000`.

Your submission must include the necessary `Dockerfile` and/or `docker-compose.yml` files.

6 Orchestrator

The orchestrator is responsible for:

- Receiving incoming requests from users.
- Tracking which worker is free or busy.
- Forwarding requests to available workers.
- Returning worker responses to users.
- Queueing requests when both workers are busy.
- Sending queued requests when a worker becomes free.

You may use a JSON file such as `status.json` to track worker state:

```
{
  "worker1": {
    "ip": "public_ip_of_worker1",
    "port": "5000",
    "status": "free"
  },
  "worker2": {
    "ip": "public_ip_of_worker2",
    "port": "5000",
    "status": "busy"
  }
}
```

Listing 1: Example worker status file

Your orchestrator must expose at least the following endpoint:

```
@app.route("/new_request", methods=["POST"])
def new_request():
    incoming_data = request.json
    # Forward the request to a free worker or queue it.
```

6.1 Queue Requirement

If both workers are busy, the orchestrator must place the incoming request in a queue.

As soon as a worker becomes free, the orchestrator must send the next queued request to that worker.

Simply storing requests in a list without eventually processing them does **not** satisfy this requirement.

7 Worker Application

Each worker must run a Flask application inside a Docker container. The worker must accept POST requests on port `5000` and return a valid JSON response.

7.1 Machine Learning Inference

You may use a lightweight Hugging Face model such as DistilBERT. Your worker should accept text input, run inference, and return the result as JSON.

7.2 Approved Fallback

If your EC2 instance type or AWS Lab environment cannot support PyTorch or the required ML libraries, you may replace ML inference with a simple text-processing function, such as reversing the input text or counting vowels.

The fallback is allowed only if:

- You clearly explain the resource limitation in your report.
- The worker still receives JSON input.
- The worker returns valid JSON output.
- The orchestrator and queueing logic still work correctly.

8 Automation and Infrastructure as Code

Your solution must be automated end to end. During the demo or evaluation, you should be able to deploy and test the system using a simple script or a small set of clearly documented scripts.

Your automation should be able to:

- Launch or identify the required EC2 instances.
- Install Docker and required dependencies.
- Copy or retrieve your application code.
- Build and start the Docker containers.
- Start the orchestrator.
- Send test requests.
- Display or save the responses.

You may use Bash, Python, Ansible, Terraform, the AWS CLI, or AWS SDKs such as `boto3`. Your chosen approach must work in the AWS Lab environment.

Include a `README.md` that explains:

- How to launch the system.
- How to run the automation scripts.
- Expected runtime.
- How to send a test request.
- How to stop and clean up AWS resources.

9 Report Requirements

Submit one PDF report per group, preferably generated using LaTeX.

Your report must include:

- Names and student IDs of all group members.
- A description of your AWS setup.
- A simple architecture diagram.
- An explanation of your Docker and Docker Compose configuration.
- An explanation of how the orchestrator selects free workers.
- An explanation of how queued requests are handled.
- A description of the worker application and ML inference or approved fallback.
- At least one example request and response.
- Instructions to run your code.
- Any assumptions, limitations, or problems encountered.

10 Submission

Submit exactly one `.zip` file named:

```
assignment2.zip
```

The archive must contain:

- `report.pdf`
- All Python source code.
- Dockerfiles and/or Docker Compose files.
- Automation scripts.
- `README.md`
- Any required configuration files.

Do not include AWS credentials, virtual environments, large model files, or unnecessary datasets.

11 Grading Rubric

This assignment is worth **100 points**.

Component	Criteria	Points
Docker setup	Dockerfiles and/or Docker Compose are correct; containers build and run successfully on the worker instances.	20
Orchestrator logic	The orchestrator receives requests, reads and updates worker status, routes requests to free workers, and correctly processes queued requests.	30
Worker application	Worker containers receive requests on port 5000 and return valid JSON using ML inference or an approved fallback.	20
Automation and IaC	The system can be deployed and tested using scripts and AWS tools with minimal manual intervention.	15
Report and code quality	The report is clear and complete; the architecture is explained; code is organized and sufficiently commented.	15
Total		**100**

11.1 Grading Notes

- All solutions should run on AWS cloud environment.
- We will use the same strategy to control for group participation by tracking each member's contributions.
- Full orchestrator points require correct queue processing, not only request forwarding.
- The ML fallback receives full worker credit only when the resource limitation is clearly justified and the JSON workflow is correct.
- Submissions containing AWS credentials will receive a penalty and may be reported under the course academic-integrity policy.

12 Academic Integrity

You may discuss the assignment with other students, but your group must write its own code, scripts, report, and analysis. Cite any external documentation, tutorials, code snippets, or AI tools used, in accordance with the course policy.